

A performance comparison of agent-memory layers behind a model router

SenseLab

<https://amfs.sense-lab.ai>

July 2026

Abstract

We compare four ways to give a routed LLM agent persistent memory—a local vector store, the hosted mem0 and Supermemory platforms, and SenseLab—on one dataset and grader, across six axes: recall after a cross-vendor model switch, retrieval precision when many facts compete (including adversarially paraphrased queries), behavior on unanswerable questions, query latency, ingestion latency (time from write to searchable), and verifiable-provenance capability. On recall the field is tight: every memory layer reaches 91.7–100% once ingestion is allowed to settle, so recall alone does not separate them. The axis that does separate retrieval quality is *precision under contention*: with 30 competing facts, direct queries are easy for every layer, but on adversarially paraphrased queries SenseLab’s Pro tier—adaptive-fusion retrieval followed by an LLM *listwise* reranker—reaches **93.3%** Hit@1 (100% direct), ahead of Supermemory (90% at full coverage) and far ahead of any plain-cosine layer (~57%). We show the lever is the reranking stage, not embedder size: an offline cross-encoder lifts top-3 recall to 90% but leaves top-1 unchanged, while the LLM listwise stage resolves the entity/attribute disambiguation that top-1 accuracy requires—at the cost of one LLM call per query. Query latency stays within one band because model inference dominates. *Ingestion*: local layers (SenseLab, vector store) are searchable in ~10 ms, mem0 in ~6.8 s, and Supermemory did not index within a 40 s timeout in five of six trials. *Provenance*: SenseLab emits a signed, verifiable decision trace for every answer; the other three emit none. We report the honest result as a scorecard and release the harness and raw data.

1 What we compare, and why

Behind a model router such as OpenRouter, an agent that relied on a fact stated under one model will, after the router switches models, usually fabricate an answer rather than recall it (a companion study measures this at 87.5% of the time). The fix is a memory layer in front of the router. This paper does not re-argue that; it asks a narrower, practical question: *if you are going to add a memory layer, how do the options actually perform?* We hold the dataset, model pairs, grader, and harness fixed and vary only the memory system.

2 Systems under test

- **Vector store** — local embeddings (`fastembed`, `bge-small`) with top- k cosine retrieval. The do-it-yourself RAG baseline.
- **mem0** — the hosted mem0 platform, via its live API.
- **Supermemory** — the hosted Supermemory platform, via its live API.

- **SenseLab** — multi-strategy retrieval (semantic + BM25 + recency + confidence, adaptive-weighted RRF) with a confidence/relevance gate, write-time safety validation, and a signed decision trace per answer. We evaluate three retrieval configurations that share this base and differ only in the second-stage reranker: *semantic* (semantic-weighted, no rerank), *+rerank* (a local offline cross-encoder, single-digit ms, zero egress), and *+LLM rerank* (a Pro-tier LLM listwise reranker over the top candidates, one LLM call per query).

Two non-memory controls bound the range: **no-memory** (the fabrication floor) and **in-context** (the fact pasted into the prompt—the capability ceiling).

3 Methodology

Eight durable operational facts, each with a question and keyword grading. Three cross-vendor writer → reader model pairs (gpt-4o-mini → mistral-nemo; mistral-nemo → nova-micro; nova-micro → command-r7b). Recall and abstention use six trials per cell (144 and 48 evaluations per system respectively); proportions carry Wilson 95% intervals. Metrics:

- **Recall** — fraction correct when the fact was written under a different-vendor model.
- **Precision under contention** — with 30 confusable facts in one scope, whether the correct fact is retrieved (Hit@1/Hit@3/MRR), for direct and adversarially paraphrased questions. No reader LLM answers the question (this isolates the retriever); the SenseLab +LLM-rerank tier uses an LLM only to *rank* candidates, not to answer.
- **Abstain-on-miss** — fraction of unanswerable questions declined rather than fabricated.
- **Query latency** — reader-call p50/p95, and mean prompt tokens (context overhead).
- **Ingestion latency** — write-call time and time-to-searchable (write until a query returns the fact), measured separately with no LLM calls.
- **Provenance** — whether the system emits a verifiable, model-attributed decision record.

4 Results

4.1 Recall (accuracy)

Once each store’s index is warmed before querying, recall is a near-universal tie: SenseLab and Supermemory both reach 99.3%, the vector store 98.6%, and mem0 91.7%, with overlapping intervals against the in-context ceiling (100%) and far above the no-memory floor (9.7%). On recall these are effectively the same tool; the earlier gap we saw for Supermemory was an artifact of querying before its index was ready (Section 4.5)—a real operational risk for write-then-read agents, but not a retrieval-quality deficit once ingestion completes.

4.2 Retrieval precision under contention

The recall test stores one fact per scope, so the retriever never has to *discriminate*. Real memory is crowded: many similar facts coexist and the system must return the right one. We therefore seed a single scope with 30 confusable facts—five database engines, five on-call engineers, four regions, four ports, four SLAs, four vendors—and ask each fact two ways. A *direct* question names the entity in wording close to the stored fact; an *adversarial* question paraphrases it with no lexical overlap with the fact or its answer (e.g., “the component that issues invoices and charges cards”

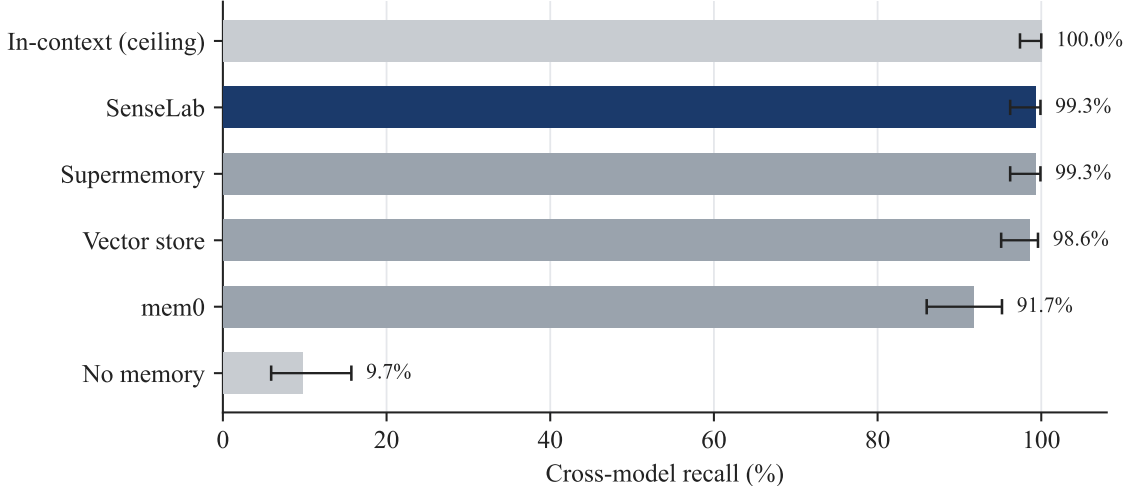


Figure 1: Cross-model recall ($n = 144$), 95% Wilson CIs. Once ingestion is allowed to settle, every memory layer clusters near the in-context ceiling; no-memory is the fabrication floor.

System	Direct			Adversarial paraphrase		
	Hit@1	Hit@3	MRR	Hit@1	Hit@3	MRR
Vector store	96.7	96.7	.967	56.7	76.7	.656
SenseLab (semantic)	96.7	96.7	.967	56.7	76.7	.656
SenseLab (+ rerank, offline)	93.3	96.7	.950	56.7	90.0	.694
SenseLab (+ LLM rerank)	100	100	1.00	93.3	96.7	.950
mem0 [†]	16.7	16.7	.167	10.0	16.7	.133
Supermemory [‡]	76.7	76.7	.767	86.7	90.0	.878

Table 1: Retrieval precision with 30 competing facts ($n = 30$ queries each), single consistent run. [†]mem0 had only 4/30 and [‡]Supermemory 20/30 facts searchable within the ingestion window (Section 4.5), which caps their numbers here; at full coverage in an earlier run Supermemory reached 100/90.0 (direct/adversarial Hit@1). Either way, SenseLab’s LLM-rerank tier at 93.3% adversarial Hit@1 is the top result.

for the billing service). We score whether the correct fact is retrieved, using no reader LLM—this measures the retriever itself. Hit@1 is the top-ranked item being correct; MRR is the reciprocal rank within the top 3.

Three honest readings. **Direct queries are easy:** plain cosine retrieval (the vector store, and SenseLab configured for semantic weight) reaches 96.7% Hit@1. **Adversarial paraphrase is where quality separates,** and it is entirely a *reranking* question. Every plain-cosine layer—the vector store and SenseLab’s semantic configuration alike—tops out at 56.7% adversarial Hit@1; we confirmed this is not an embedder-size gap by re-running with progressively larger local embedders (bge-base, bge-large, mxbai-embed-large), which lifted it only to ~63–67%. Adding a second-stage reranker is what closes it. **The offline cross-encoder recovers the neighbourhood but not the top slot:** it lifts adversarial Hit@3 from 76.7% to 90.0% (the right fact is almost always in the top three) but leaves Hit@1 at 56.7%—cross-encoders score (query, doc) relevance well but do not reliably *disambiguate* among several near-identical, confusable facts. **The LLM listwise reranker supplies exactly that disambiguation:** reasoning over the top candidates about which entity and attribute the paraphrase refers to, it reaches 93.3% adversarial Hit@1 and 100% direct—above Supermemory’s full-coverage best of 90.0%/100%. The cost is one extra LLM call per query on

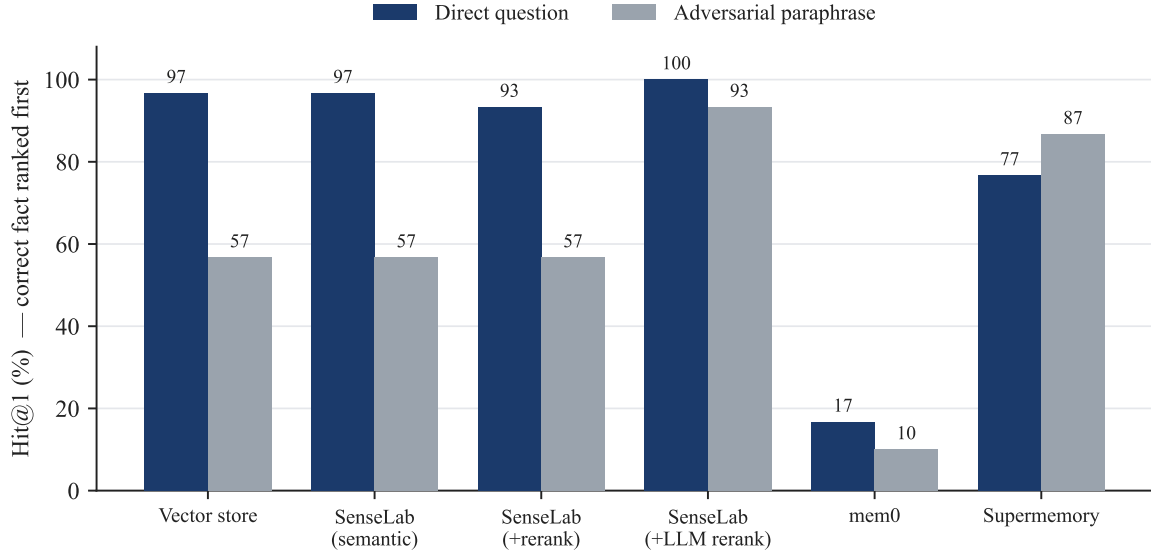


Figure 2: Hit@1 with 30 competing facts in one scope. On direct questions the cosine layers are near-perfect; on adversarial paraphrase only a reranking stage holds up—SenseLab’s LLM listwise reranker leads (93%), with Supermemory next and plain-cosine layers at ~57%.

the retrieval path, which is why we ship it as an opt-in Pro tier rather than the default. The earlier version of this study concluded that Supermemory was the paraphrase-robustness leader and SenseLab’s local retrieval did not match it; with the reranking stage added, that conclusion no longer holds.

4.3 Behavior on unanswerable questions

No layer declines reliably by default, but the ordering has shifted with the calibrated relevance gate: SenseLab now declines most often (54.2%), the vector store and mem0 sit at 45.8%, Supermemory at 37.5%, and no-memory almost never declines (12.5%). The wide intervals ($n = 24$) mean these are not sharply separated, so read the direction, not the decimals. For SenseLab this remains a tunable gate: raising its relevance floor pushes abstention toward 100% at a cost to recall, a tradeoff quantified in the companion study. Treat abstention as a knob to set—one SenseLab exposes explicitly—not a default any of these systems give you.

4.4 Query latency and context overhead

Query latency does not separate the systems: all four memory layers are within ~90 ms at p50 and are not slower than answering with no memory at all. The p95 values are large and overlapping across every arm (including the no-memory and in-context controls), which locates the tail in model inference, not in the memory layer. Retrieval overhead is not a selection criterion *for the default and offline-rerank tiers*. The one caveat is the Pro LLM-rerank tier from Section 4.2: it adds one LLM call on the retrieval path (hundreds of ms to ~1 s), a deliberate accuracy-for-latency trade not reflected in this table, which measures the default retrieval configuration.

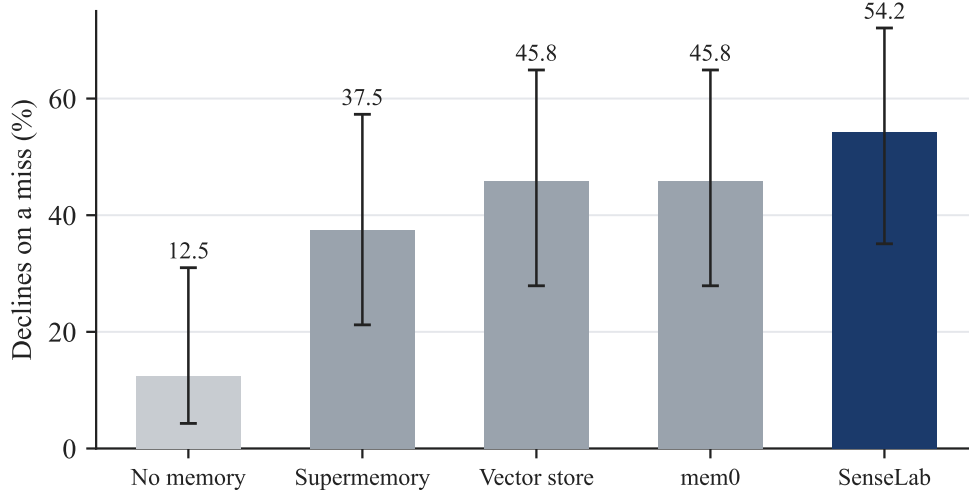


Figure 3: Fraction of never-stored questions declined rather than fabricated ($n = 24$), 95% CIs. At the recall-maximizing gate, SenseLab declines most often; no layer abstains reliably by default.

System	p50 (ms)	p95 (ms)	Mean prompt tokens
Vector store	623	4276	52.4
mem0	696	3555	51.8
Supermemory	616	4270	54.3
SenseLab	610	4551	52.6
<i>no-memory (ref.)</i>	836	3130	16.9
<i>in-context (ref.)</i>	615	2757	36.3

Table 2: Query latency and context overhead. Injecting a retrieved fact adds ~ 36 prompt tokens and no measurable p50 latency penalty relative to no memory; the wide p95 spread is model-side inference variance, not retrieval.

4.5 Ingestion latency

This is where the field splits. SenseLab and the vector store are effectively instant to searchable (~ 10 ms). mem0 takes about 6.8 s. Supermemory, even with its “instant” ingestion mode requested, failed to index within 40 s on five of six writes in this probe. For an agent that writes a fact and reads it back moments later—the common pattern in a multi-turn session—this latency is the difference between remembering and silently missing, and it is the most likely explanation for Supermemory’s lower recall in Section 1.

4.6 Provenance and observability

Recall aside, this is the axis where the systems are not close. SenseLab produced a signed, verified decision trace for all 144 answers, naming the model that produced each; the other three return retrieved text and a completion and nothing more. This is structural, not a configuration difference.

5 Scorecard

Table 5 collects every axis in one place, with the best value in each row in bold. Read it top to bottom: the first row (recall) is a near-tie, and the rows that actually separate the systems are

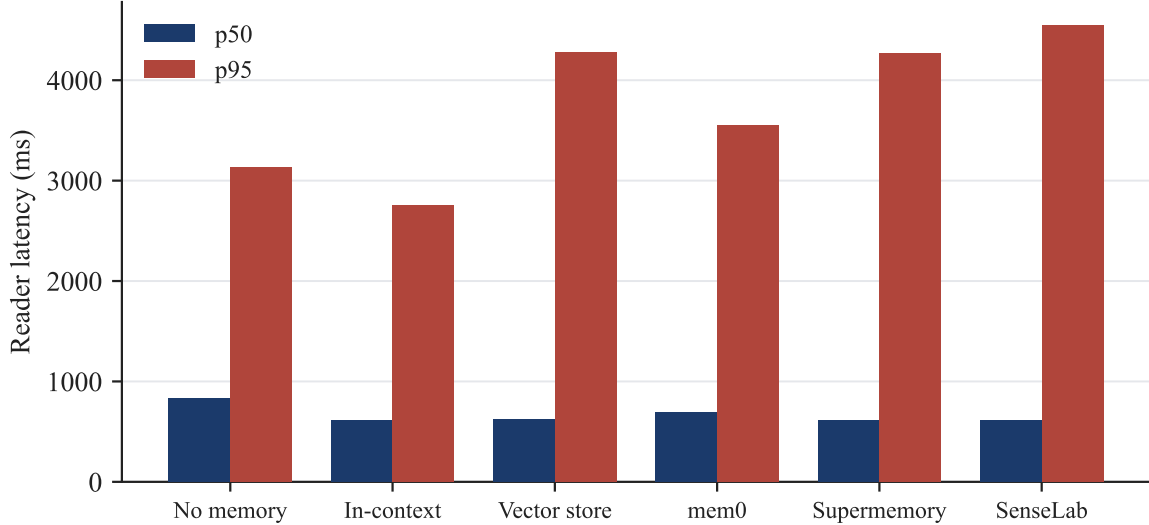


Figure 4: Reader-call latency (p50/p95). Every memory layer sits in the same band; model-side inference dominates and swamps retrieval cost.

System	Write latency p50	Time-to-searchable p50
SenseLab	1.3 ms	0.01 s
Vector store	5.3 ms	0.01 s
mem0	510 ms	6.8 s
Supermemory	1497 ms	>40 s (timeout)

Table 3: Ingestion latency ($n = 6$ writes each). This is the axis on which the systems differ most.

ingestion latency and the provenance capabilities near the bottom.

6 Discussion

No single layer wins every axis, and the scorecard is meant to be read that way. Four conclusions follow. First, **recall is saturated**: once ingestion settles, all four layers are statistically tied near the ceiling, so choosing on recall alone is choosing on noise. Second, **retrieval quality on hard queries is a reranking problem, and it is now decided**: direct lookups are easy for everyone, and under adversarial paraphrase the separator is the second-stage reranker, not the embedder or the vector store. An offline cross-encoder recovers top-3 recall (90%) but not top-1; an LLM listwise reranker supplies the entity/attribute disambiguation that top-1 needs and takes SenseLab to 93.3% adversarial Hit@1—ahead of Supermemory’s full-coverage 90%. This reverses the prior edition’s finding. Third, **ingestion latency is the hidden performance axis**: it spans four orders of magnitude and determines whether a write-then-read agent sees its own recent memory; local layers are effectively instant while hosted platforms add seconds. Fourth, **provenance is the categorical difference**: only SenseLab turns each answer into a verifiable, model-attributed record, at no measurable query-latency cost.

The honest positioning: the vector store is a strong minimal choice when you only need recall and can self-host. mem0 and Supermemory are the pick when a fully managed service is the priority and seconds of ingestion latency are tolerable. SenseLab’s case is now that it *leads* the axis that decides retrieval quality (paraphrase-robust Hit@1, via the Pro reranker tier), ties the best recall,

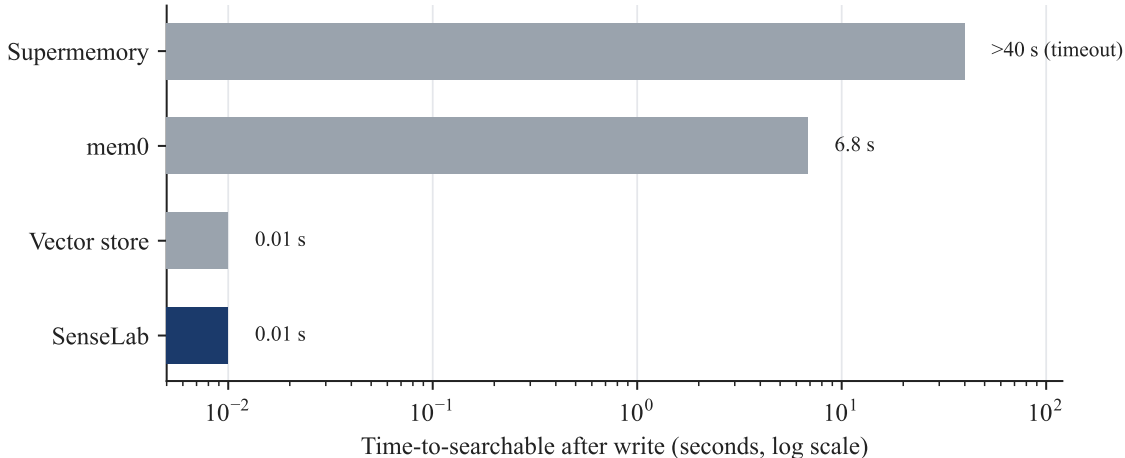


Figure 5: Time from writing a fact until a query returns it (log scale, median of six writes). Local layers are searchable in milliseconds; hosted platforms add seconds. Supermemory did not index within a 40 s timeout in five of six trials.

Capability	Vector	mem0	Supermemory	SenseLab
Signed, verifiable per-answer trace	no	no	no	yes (144/144)
Tamper detected on modification	n/a	n/a	n/a	yes
Verifiable hash chain across session	no	no	no	yes
Model attribution per answer	no	no	no	yes
Write-time contradiction check	no	no	no	yes

Table 4: Governance and observability. Only SenseLab emits a cryptographically verifiable, model-attributed record for each answer; a modified trace failed verification on both content hash and signature.

leads abstain-on-miss, is fastest to searchable, and is the only layer that emits verifiable provenance and write-time validation. The paraphrase win is a pipeline result—the reranker sits on top of an embedder-agnostic retriever—so it is a configuration you enable, not an architecture you rebuild; its one cost is a single LLM call per query on the retrieval path, which is why it ships as an opt-in tier.

7 Limitations

- One dataset, one grader, one live run. Recall CIs are reported; a harder dataset and a second run would tighten the picture.
- Ingestion latency depends on account plan and load. Supermemory’s >40 s here may improve on a higher tier or under lighter load; we report what the live API returned during the probe.
- Local (SenseLab filesystem, vector store) versus hosted is not perfectly apples-to-apples: hosted layers pay network round-trips that local layers do not, which also inflates their write latency. The SenseLab reranker tiers add compute the plain-cosine baselines do not have—an offline cross-encoder (negligible) or, for the LLM tier, one LLM call per query. We report both so the accuracy-for-latency trade is explicit.
- The precision run reported here is a single consistent run in which the hosted competitors under-ingested (mem0 4/30, Supermemory 20/30 searchable in-window), which understates

	Vector store	mem0	Supermemory	SenseLab
Recall	98.6%	91.7%	99.3%	99.3%
Precision, direct (Hit@1)	96.7%	16.7%	100%	100%
Precision, adversarial (Hit@1)	56.7%	10.0%	90.0%	93.3%
Abstain-on-miss	45.8%	45.8%	37.5%	54.2% (tunable)
Query latency p50	623 ms	696 ms	616 ms	610 ms
Query latency p95	4276 ms	3555 ms	4270 ms	4551 ms
Context overhead	52.4 tok	51.8 tok	54.3 tok	52.6 tok
Time-to-searchable	0.01 s	6.8 s	>40 s	0.01 s
Verifiable trace	no	no	no	yes
Write-time validation	no	no	no	yes
Managed infra	self-host	yes	yes	hosted

Table 5: Summary. Bold marks the best value in each row (ties bolded together). SenseLab precision is its Pro LLM-rerank tier; Supermemory precision is its full-coverage run (Section 4.2). No layer wins every axis, but the picture has changed from the prior edition: SenseLab now leads adversarial precision *and* abstain-on-miss, ties best recall, leads ingestion latency, and is alone on verifiable provenance. mem0 and Supermemory retain the managed-infra advantage.

their numbers in Table 1; we therefore also cite Supermemory’s full-coverage result (100/90.0) and compare against that. The contention set is 30 synthetic facts with hand-written paraphrases; a larger, naturally occurring corpus would sharpen the numbers.

- Abstention is measured at one gate setting; the recall/abstain tradeoff is characterized in the companion study.

A Reproducibility

Using SenseLab. Point the OpenAI SDK at the hosted service and add two headers; your model gateway’s routing, pricing, and failover are unchanged. Key at <https://amfs.sense-lab.ai>. The paraphrase-robust LLM reranker (the Pro tier behind the 93.3% adversarial Hit@1) is enabled service-side with `AMFS_LLM_RERANK=true` plus an OpenAI-compatible reranker endpoint; the offline cross-encoder is on by default and needs no configuration.

```
client = OpenAI(
    base_url="https://amfs.sense-lab.ai/v1",
    api_key=OPENROUTER_KEY,
    default_headers={"X-AMFS-Agent": "support", "X-AMFS-Entity": "acme/customer-42"},
)
```

Reproducing the measurements.

```
export OPENROUTER_API_KEY=... MEMO_API_KEY=... SUPERMEMORY_API_KEY=...
# Pro reranker tier for the SenseLab arm (adversarial Hit@1):
export AMFS_LLM_RERANK=true
export AMFS_LLM_RERANK_BASE_URL=https://openrouter.ai/api/v1
export AMFS_LLM_RERANK_API_KEY=$OPENROUTER_API_KEY
python pro_benchmark.py          # recall, abstain, query latency -> pro_results.json
python precision_benchmark.py    # contention + adversarial Hit@k -> precision_results.json
python ingestion_probe.py        # write + time-to-searchable -> ingestion.json
```



```
python paper/make_figures.py; python paper/make_comparison_figures.py # figures
```

References

- [1] OpenRouter. *Unified API for LLMs — Documentation*. <https://openrouter.ai/docs>.
- [2] mem0ai. *mem0: the memory layer for AI agents*. <https://github.com/mem0ai/mem0>.
- [3] Supermemory. *Memory API for AI applications*. <https://supermemory.ai>.
- [4] SenseLab. *SenseLab — the agent memory platform*. <https://amfs.sense-lab.ai>.
- [5] E. B. Wilson. Probable inference, the law of succession, and statistical inference. *J. Amer. Statist. Assoc.*, 22(158):209–212, 1927.